93.8 Pass

Siba Prasad Mangal

PDF generated at: 11 Feb 2026 18:55:17 UTC

View this report on HackerRank

Score

93.8% • 187.5 / 200

scored in eClerx - Java Coding GCC Test - 4+ Years in 43 min 37 sec on 11 Feb 2026 23:38:34 IST

Candidate Information

Email	siba.mangal25@gmail.com
Test	eClerx - Java Coding GCC Test - 4+ Years
Candidate Packet	View
Taken on	11 Feb 2026 23:38:34 IST
Time taken	43 min 37 sec/ 136 min
Work Experience	3 years
Invited by	Vipul
Invited on	6 Feb 2026 23:04:10 IST

Skill Distribution

No.	Skill	Score
1	Java Intermediate	92%
2	SQL Intermediate	66%
3	Java Basic	100%

4	Spring Boot Intermediate	100%

Tags Distribution

Medium	93%	Collections	100%
Java	95%	Queues	100%
Stacks	100%	Garbage Collection	50%
Exception Handling	50%	Programming	50%
MySQL	66%	Constraints	0%
Easy	100%	Diode	100%
Semiconductor	100%	Lambda Expression	100%
ORM	100%	Spring MVC	100%
CRUD Operations	100%	Hibernate	100%
Spring Boot	100%		

Questions

Java (Intermediate) • 17.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Empty Collection Multiple Choice	Java (Intermediate)	5/5	-	3 min 28 sec	
2	Java Garbage Collection Multiple Choice	Java (Intermediate)	2.5/5	-	1 min 28 sec	

3	Identify Runtime Exceptions Multiple Choice	Java (Intermediate)	5/5	-	1 min	✓
4	Look Up an Element in a Map Using a Key Multiple Choice	Java (Intermediate)	5/5	-	27 sec	✓
5	Root Class of the Java Exception Heirarchy Multiple Choice	Java (Intermediate)	0/5	-	36 sec	✗

SQL (Intermediate) • 20 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	MySQL: FOREIGN KEY Constraint Multiple Choice	SQL (Intermediate)	0/5	-	1 min 8 sec	✗
7	MySQL: Substring Function Multiple Choice	SQL (Intermediate)	5/5	-	1 min 14 sec	✓
8	MySQL: Group By Order Multiple Choice	SQL (Intermediate)	5/5	-	1 min 25 sec	✓
9	Java Types Multiple Choice	Java (Basic)	5/5	-	1 min 17 sec	✓
10	Diode Basics and Equivalent Circuits Multiple Choice	-	5/5	-	48 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
11	Java: Inventory Balance Enquiry Coding	Java (Intermediate)	75/75	-	9 min 54 sec	✓

Spring Boot (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
12	Java SpringBoot: School API Back-end Developer	Spring Boot (Intermediate)	75/75	-	20 min 6 sec	✓

1. Empty Collection

Correct

Multiple Choice | Medium | Collections | Java | Queues | Stacks

Question description

Which code displays an empty (`[]`) collection when executed?

Interviewer guidelines

The correct answer is D.

There are 10 items in the stack initially. In the first loop, we add these 10 items to the stack. In the second loop, we iterate based on the size of the stack, and in every iteration, we remove the item from the top of the stack and insert it into the queue. Then decrement the counter. This is a very important step as this will adjust the index counter and will remove all the items from the stack.
For example, if i is not decremented, then the stack size is reduced and i is incremented. After 4th iteration, the value of the variable I will be greater than the size of the stack and exit from the loop.
But the stack will not be empty. The final elements in the stack are mentioned below:

[5, 6, 7, 8, 9]

To avoid this, we need to decrement the value of the variable i so that it will never increment. It will be always '0'. But the loop will exit when the stack becomes empty as the condition is evaluated to false.

The same case is applicable to the third loop as well as the variable i is required to be reduced. This way it will remove all the items from the queue.

Based on the above explanation, other answers are not correct.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size() - 1; i++) { int n = stack.remove(0); queue.add(n); } for (int i = 0; i < queue.size() - 1; i++) { int n = (int) queue.remove(); i = i - 1; } System.out.println(queue);</code></pre>

- <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); } System.out.println(queue); </code></pre>

- <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); i = i - 1; } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); } System.out.println(queue); </code></pre>

- **<pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); i = i - 1; } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); i = i - 1; } System.out.println(queue); </code></pre>   **

No comments.

2. Java Garbage Collection

Partially correct

Multiple Choice | Garbage Collection | Medium

Question description

Which of the following statements are true about garbage collection in Java?

Interviewer guidelines

When an object is created using the new keyword, it is allocated on the heap memory, and the garbage collector only runs for the heap memory. On the other hand, static variables in Java are allocated in the stack space, which is separate from the heap. Calling the System.gc() does not guarantee that garbage collection will occur in Java.
This method is used to suggest to the JVM that a garbage collection cycle should be performed. However, the JVM has its own algorithm for deciding when to perform garbage collection, based on some factors. When an object is no longer reachable by the program, the garbage collector will eventually reclaim its memory. Before this happens, the garbage collector will call the object's finalize() method (if it is overridden) as part of the finalization process. If that method restores the object by making it reachable again, then the object will not be garbage collected. However, It's important to note that the finalize() method is not always a reliable mechanism for restoring an object or preventing it from being garbage collected.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] The garbage collector can only free memory that was allocated using the new keyword.
- [ ] The garbage collector can free memory that is being referenced by a static variable.
- [ ] The system.gc() method can be used to guarantee that garbage collection will occur.
- [x] The finalize() method can be used to prevent an object from being garbage collected.

No comments.

3. Identify Runtime Exceptions

Correct

Multiple Choice | Java | Exception Handling | Programming | Medium

Question description

Which of the following is **not** an example of RuntimeException ?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- NullPointerException
- ArrayIndexOutOfBoundsException
- **FileNotFoundException**

No comments.

4. Look Up an Element in a Map Using a Key

Correct

Multiple Choice | Java | Collections | Medium

Question description

Which of the methods would you use to look up a element in a given map using a specified key.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] get()
- [ ] look()
- [ ] search()
- [ ] value()

No comments.

5. Root Class of the Java Exception Heirarchy

Incorrect

Multiple Choice | Java | Medium | Exception Handling | Programming

Question description

Which of the following is the root class (apex) of the exception heirarchy in Java?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] Throwable
- [x] Exception
- [ ] Error

RuntimeException

No comments.

6. MySQL: FOREIGN KEY Constraint

Incorrect

Multiple Choice | MySQL | Medium | Constraints

Question description

Which expression is not correct?

Interviewer guidelines

From the MySQL Reference Manual for foreign key constraints:

- MySQL supports foreign key references between one column and another within a table.
- A column cannot have a foreign key reference to itself.
- Corresponding columns in the foreign key and the referenced key must have similar data types.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- A column might have a foreign key reference to itself.
- MySQL supports foreign key references between one column and another within a table.
- Corresponding columns in the foreign key and the referenced key must have similar data types.
- **None of the above, all expressions are correct.**

No comments.

7. MySQL: Substring Function

Correct

Multiple Choice MySQL Medium

Question description

Select the MySQL expression that returns a different value from the others.

Interviewer guidelines

All expressions return @

Candidate's Solution

Options: (Expected answer indicated with a tick)

- SELECT SUBSTR('user@domain.com', 5, 1)
- SELECT SUBSTR('user@domain.com' FROM 5 FOR 1)
- SELECT SUBSTR('user@domain.com' FROM -11 FOR 1)
- None of the above, all expressions return the same value.

No comments.

8. MySQL: Group By Order

Correct

Multiple Choice MySQL Medium

Question description

Which expression causes a MySQL error?
Select all that apply.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ SELECT is_active, COUNT(*) FROM transactions GROUP BY is_active ORDER BY 2
- ○ SELECT is_active, COUNT(*) FROM transactions GROUP BY is_active ORDER BY COUNT(*)
- ○ SELECT is_active, COUNT(*) AS transactions FROM transactions GROUP BY is_active ORDER BY transactions
- ● none of the above, all expressions are correct ✓

No comments.

---

9. Java Types

Correct

Multiple Choice | Easy | Java

Question description

Which of the following is not a Java primitive type?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] String
- [ ] float
- [ ] double
- [ ] int
- [ ] byte

No comments.

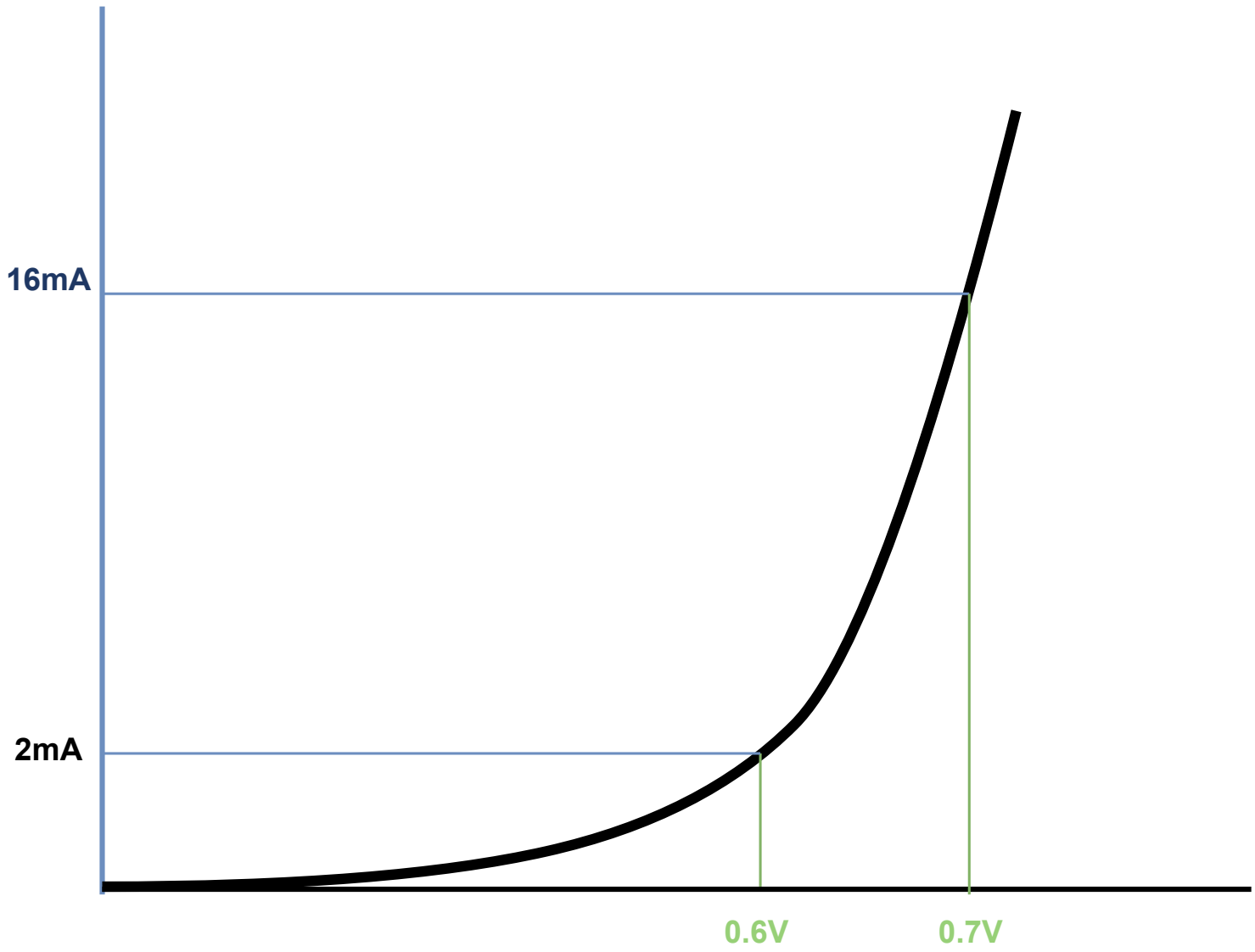
10. Diode Basics and Equivalent Circuits

Correct

Multiple Choice | Easy | Diode | Semiconductor

Question description

An engineer is developing a sensor array for an embedded system that includes a new silicon diode among its components. The system relies on precise readings from this array to make real-time decisions through its software. To ensure the diode's performance aligns with the system's requirements, the engineer is provided with the I-V characteristics curve of the diode. The task is to approximate the average AC resistance, r_{av}, over a specified segment of this curve. This resistance calculation is critical as it will directly influence the logic in the system's software, affecting how the embedded system interprets sensor data and makes operational decisions. Using just the curve for reference, determine the *rav* between the marked points. What is the estimate?



Interviewer guidelines

r_{av} is the average AC resistance found between two marked points in the I-V graph. Here r_{av}=0.1/14mA=7Ω.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Not enough details
- 11.2 Ohms

6 Ohms

7 Ohms

No comments.

11. Java: Inventory Balance Enquiry

Correct

Coding Medium Lambda Expression Java

Question description

In an e-commerce warehouse, keeping track of the available inventory for each product is crucial for fulfilling orders efficiently. The system should be able to handle queries about the current inventory levels and alert if any product's stock falls below a safety threshold, indicating that it might soon be out of stock.

Create a system that can quickly report the current inventory status of a list of products represented by unique identifiers.

Write a method *findProductsBelowThreshold*, that takes a list of product IDs of size *n* and a safety stock threshold as inputs, then returns a list of product IDs with stock levels below the specified threshold.

For each product ID, check the current stock level. If the stock level is below the safety threshold, add it to the result list. There is an anonymous method *getCurrentStockLevel* that takes a product ID and returns the current stock level for that product.

Implement the classes shown.

- Create a class called *StockLevelQuery* that implements the *IStockLevelQuery* interface.
  - This class should have the following properties:
    - *Map<String, Integer> stockLevels*
  - Implement the constructor *StockLevelQuery(Map<String, Integer> stockLevels)*
  - Create a method called *getCurrentStockLevel(String productId)* that
    - returns *stockLevels* that represent the current stock level for that product
- Create a class called InventoryChecker.

- Implement the constructor *InventoryChecker(IStockLevelQuery stockLevelQuery).*
- Create a method called *findProductsBelowThreshold(List<String> productIds, int threshold)* that
  - returns a list of strings representing the product IDs with stock levels below the threshold.

Function Description

Complete the classes *StockLevelQuery* and *InventoryChecker* in the editor below.

Constraints

- $1 \le n \le 1000$
- $1 \le productIds[i].length \le 20$
- *productIds[i]* contains only uppercase English letters and digits.
- $1 \le threshold \le 1000$
- $0 \le$ return value of *getCurrentStockLevel()* ≤ 1000.

▼ INPUT FORMAT FOR CUSTOM TESTING

For each test case, the input follows this format:

- A line containing a single integer *n*, the number of product identifiers.
- A line with *n* space-separated strings, each representing a product identifier.
- A line containing a single integer *threshold*, the safety stock threshold for the products.
- *n* lines, each containing a string *productId* followed by an integer *stockLevel*, that represents the stock level for the given product identifier. Each pair (*productID* and *stockLevel*) is separated by a space.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
3
A12B3 C4D56 E789F
50
A12B3 100
C4D56 45
E789F 60
```

Sample Output

```
C4D56
```

Explanation

There are three product identifiers with a safety stock threshold of 50. For each product identifier, the stock level is provided as follows:

- “A12B3” has a stock level of 100, which is above the threshold and, therefore, not included in the output.
- “C4D56” has a stock level of 45, which is below the threshold, so it is included in the output.

- “E789F” has a stock level of 60, which is above the threshold, so it is not included in the output.

The output is “C4D56” since it is the only product identifier with a stock level below the threshold.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
4
G10H11 XYZ123 ABC789 PQR456
100
G10H11 20
XYZ123 150
ABC789 80
PQR456 90
```

Sample Output

```
G10H11
ABC789
PQR456
```

Explanation

There are four product identifiers and a threshold of 100. The stock levels are as follows:

- “G10H11” has a stock level of 20, which is below the threshold, so it is included in the output.
- “XYZ123” has a stock level of 150, which is above the threshold, so it is not included in the output.
- “ABC789” has a stock level of 80, which is below the threshold, so it is included in the output.
- “PQR456” has a stock level of 90, which is below the threshold, so it is included in the output.

The output lists “G10H11”, “ABC789” and “PQR456” as the product identifiers with stock levels below the threshold.

Interviewer guidelines

▼ SOLUTION

```
class StockLevelQuery implements IStockLevelQuery {
    private final Map<String, Integer> stockLevels;

    public StockLevelQuery(Map<String, Integer> stockLevels) {
        this.stockLevels = stockLevels;
    }

    @Override
    public int getCurrentStockLevel(String productId) {
        return stockLevels.getOrDefault(productId, 0);
    }
}

class InventoryChecker {
    private final IStockLevelQuery stockLevelQuery;
```

```
    public InventoryChecker(IStockLevelQuery stockLevelQuery) {
        this.stockLevelQuery = Objects.requireNonNull(stockLevelQuery, "stockLevelQuery must not be null");
    }

    public List<String> findProductsBelowThreshold(List<String> productIds, int threshold) {
        return productIds.stream()
            .filter(productId -> stockLevelQuery.getCurrentStockLevel(productId) < threshold)
            .collect(Collectors.toList());
    }
}
```

Candidate's Solution

Language used: **Java 8**

```
import java.util.*;
import java.io.*;
import java.util.stream.*;

interface IStockLevelQuery {
    int getCurrentStockLevel(String productId);
}
class StockLevelQuery implements IStockLevelQuery {
    private Map<String, Integer> stockLevels;

    public StockLevelQuery(Map<String, Integer> stockLevels){
        this.stockLevels = stockLevels;
    }

    @Override
    public int getCurrentStockLevel(String productId){
        if(stockLevels.containsKey(productId)){
            return stockLevels.get(productId);
        }
        return 0;
    }
}

class InventoryChecker {

    private IStockLevelQuery stockLevelQuery;

    public InventoryChecker(IStockLevelQuery stockLevelQuery){
        this.stockLevelQuery = stockLevelQuery;
    }

    public List<String> findProductsBelowThreshold(List<String> productIds, int threshold){
        List<String> result = new ArrayList<>();
```

```java

        for(String id: productIds){
            int currentStock = stockLevelQuery.getCurrentStockLevel(id);

            if(currentStock < threshold){
                result.add(id);
            }
        }

        return result;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());
        List<String> productIds = Arrays.asList(br.readLine().trim().split(" "));
        int threshold = Integer.parseInt(br.readLine().trim());

        Map<String, Integer> stockLevels = new HashMap<>();
        for (int i = 0; i < n; i++) {
            String[] productStock = br.readLine().trim().split(" ");
            stockLevels.put(productStock[0], Integer.parseInt(productStock[1]));
        }

        IStockLevelQuery stockLevelQuery = new StockLevelQuery(stockLevels);
        InventoryChecker inventoryChecker = new InventoryChecker(stockLevelQuery);

        List<String> belowThresholdProducts = inventoryChecker.findProductsBelowThreshold(productIds, threshold);
        belowThresholdProducts.forEach(System.out::println);

        br.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.1138 sec	31.3 KB
TestCase 1	Easy	Sample	Success	1	0.1154 sec	31 KB

TestCase 2	Medium	Hidden	Success	4	0.1056 sec	31 KB
TestCase 3	Medium	Hidden	Success	4	0.1168 sec	31 KB
TestCase 4	Medium	Hidden	Success	8	0.1278 sec	31.3 KB
TestCase 5	Medium	Hidden	Success	8	0.1163 sec	31.3 KB
TestCase 6	Medium	Hidden	Success	4	0.1254 sec	31 KB
TestCase 7	Medium	Hidden	Success	4	0.1241 sec	31.5 KB
TestCase 8	Medium	Hidden	Success	8	0.1255 sec	31.4 KB
TestCase 9	Medium	Hidden	Success	8	0.1272 sec	31.1 KB
TestCase 10	Easy	Sample	Success	1	0.1172 sec	30.6 KB
TestCase 11	Medium	Hidden	Success	4	0.095 sec	31 KB
TestCase 12	Medium	Hidden	Success	4	0.1255 sec	31.1 KB
TestCase 13	Easy	Hidden	Success	8	0.1215 sec	31.8 KB
TestCase 14	Medium	Hidden	Success	8	0.1192 sec	31.1 KB

No comments.

12. Java SpringBoot: School API

Correct

Back-end Developer | Medium | ORM | Spring MVC | CRUD Operations | Hibernate | Spring Boot

Question description

In this project, there is a teacher and student management system for a typical school where a teacher has many students and a student has many teachers. Here we are using a hibernate many-to-many relationship to manage such information.

Here is an example of a teacher data JSON object:

```
{
   "id": 1,
   "name": "Mark",
   "students": "[]"
}
```

Here is an example of a student data JSON object:

```
{
   "id": 1,
   "name": "Salo",
   "teachers": "[]"
}
```

The application should adhere to the following API format and response codes in SchoolController :

POST /school/teacher/{teacherId}/addStudent :

- Assume that the given *teacherId* already exists in the database.
- The request body will have a student instance. It might be a new student or an existing one.
- Assume that the given student has never been added to the given teacher.
- Return the teacher instance after adding the student, along with status code 201.

GET school/teacher/{teacherId}/students :

- Assume that the given *teacherId* already exists in the database.
- Return the list of students associated with that *teacherId* and status code 200.

GET /school/student/{studentId}/teachers :

- Assume the given *studentId* exists in the database.
- Return the list of teachers associated with the *studentId* and status code 200.

Candidate's Submission

Testcase	Test file	Status	Score
addExistingStudentToATeacher	TEST-com.hackerrank.api.controller.SchoolControllerTest.xml	Success	18.75 / 18.75
getStudentsOfATeacher	TEST-com.hackerrank.api.controller.SchoolControllerTest.xml	Success	18.75 / 18.75
addNewStudentToATeacher	TEST-com.hackerrank.api.controller.SchoolControllerTest.xml	Success	18.75 / 18.75
getTeachersOfAStudent	TEST-com.hackerrank.api.controller.SchoolControllerTest.xml	Success	18.75 / 18.75

Review logs: output log
View candidate code

No comments.